DOCUMENTACIÓN TÉCNICA DE DESPLIEGUE Proyecto de Inteligencia Artificial para Detección de Operaciones Fraudulentas PLD			
Alumno:	Miguel Ángel García Vidal		
Matrícula:	AL02825804	Versión:	1.0
Estado:	Prototipo		
Fecha:	02-Nov-2026		

1 RESUMEN EJECUTIVO

La iniciativa busca ampliar la capacidad de monitoreo PLD y reducir trabajo manual mediante un esquema híbrido de reglas, clasificación de riesgo y validación humana. La decisión PLD final no se delega completamente al modelo. La recomendación ejecutiva es aprobar únicamente un piloto controlado y decidir el escalamiento contra criterios go/no-go previamente aprobados.

2 INTRODUCCIÓN

Documento alineado con ISO/IEC 23053:2022 para describir arquitectura, operación, despliegue, monitoreo y gobierno del modelo.

2.1 PROPÓSITO DEL DOCUMENTO

Describir técnica y formalmente el despliegue del sistema de IA, incluyendo su arquitectura, componentes, procesos operativos y consideraciones de mantenimiento, conforme al marco ISO/IEC 23053:2022, que define los componentes y funciones de los sistemas de IA basados en ML. (ISO/IEC, 2022).

3 OBJETIVOS

Validar mediante un piloto controlado si una solución híbrida de IA puede aumentar la cobertura de monitoreo y reducir el esfuerzo PLD manual manteniendo trazabilidad, capacidad operativa y desempeño de detección dentro de umbrales aprobados. La fecha objetivo se definirá con stakeholders.

- Objetivo operativo vigente: liberar 90% del trabajo PLD manual, sujeto a validación en piloto.
- Las metas FP ≤2% y FN ≤1% quedan como hipótesis iniciales hasta definir denominadores y capacidad de alertas.
- Las métricas go/no-go incluirán precisión, recall, FPR, FNR, volumen de alertas y SLA.

4 ALCANCE

El alcance comprende el entrenamiento, el despliegue y la ejecución en línea del modelo de IA para la detección de operaciones fraudulentas PLD. Estos componentes se describen en el contexto de un prototipo académico y un piloto controlado, sin delegar completamente al modelo la decisión PLD final.

- **Entrenamiento:** comprende el entrenamiento del modelo Random Forest v3 para la clasificación binaria de operaciones como fraude o no fraude, utilizando el conjunto de datos académico de 51,000 registros. La validación se realiza mediante StratifiedKFold de cinco particiones. Los resultados obtenidos corresponden al ejercicio académico y no constituyen evidencia suficiente para su uso en producción.

- **Despliegue:** comprende la construcción y ejecución de un contenedor Docker que integra el entorno Python, las dependencias del servicio y el artefacto del modelo en formato PKL. El contenedor permite ejecutar la solución mediante FastAPI y Uvicorn dentro del entorno del prototipo. API**:** comprende la interfaz REST implementada con FastAPI, a través de la cual el cliente envía solicitudes al endpoint POST /predict. El flujo contempla la validación de los datos recibidos, la ejecución del modelo y la devolución de una respuesta en formato JSON.
- **Inferencia:** comprende la aplicación del modelo a los datos enviados en cada solicitud, utilizando el umbral de clasificación de 0.37 documentado para generar la respuesta. El resultado forma parte de un esquema híbrido de reglas, clasificación de riesgo y validación humana; no sustituye la decisión PLD final.

5 MARCO NORMATIVO

- ISO/IEC 23053:2022 – Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML)
- ISO/IEC 22989 (conceptos y terminología de IA)
- ISO/IEC 23894 (gestión de riesgos de IA)
- Documentación interna del proyecto: Business Case.

6 ARQUITECTURA DEL SISTEMA DE IA

6.1 ARQUITECTURA LÓGICA

Los componentes lógicos que componen la arquitectura se enlistan a continuación:

- Componentes de Operación y soporte - API REST; servicio expuesto que será el responsable de atender las peticiones de los clientes. Este cliente cuenta con validación interna para no permitir que datos no esperados ingresen al modelo de ML.
- Componentes de ML – Modelo de ML de tipo Random Forest que es el encargado de realizar la clasificación de las transacciones con la información proporcionada. El umbral utilizado para este modelo es de 0.37.
- Componentes de Datos – Dataset utilizado para el entrenamiento.

6.2 ARQUITECTURA FÍSICA

Los componentes físicos, utilizados para realizar el despliegue de la solución de ML se enlistan a continuación.

- Contenedor Docker ejecutando Python 3.13
- Web Service FastAPI, Uvicorn,
- Joblib y modelo PKL.

6.3 COMPONENTES DEL SISTEMA

Componente	Descripción
Fuente de Datos	Para este ejercicio académico se utilizó como base el Dataset IBM AMLSim y se complementó con comportamientos generados de acuerdo a simulaciones.
Procesamiento de datos	Se realizó limpieza quitando columnas que no aportan valor a la inferencia, el detalle del procesamiento del Dataset se puede revisar en el Notebook DataLoader del proyecto.
Modelo ML	Modelo de tipo RandomForest
Motor de inferencia	Componente que ejecuta la predicción; servicio Python generado, el cual invoca al modelo en su formato pkl.
API	Se utilizó FastAPI en Python para generar el API que atenderá las solicitudes del cliente.
Infraestructura	Contenedor Docker cuya configuración se encuentra en el archivo Dockerfile.

7 DATOS

El Dataset utilizado para el entrenamiento después de la limpieza consta de 51,000 registros.

Las variables de entrada para lograr la inferencia son:

- initial_balance – Balance inicial de la cuenta.
- tx_amount – Monto de la transacción.
- client_income – Ingresos declarados del cliente.
- historical_amount – histórico promedio del monto de transacciones del cliente.
- incorparation_days – Número de días que tiene como cliente.

La variable de salida es la predicción con un valor booleano de True o False.

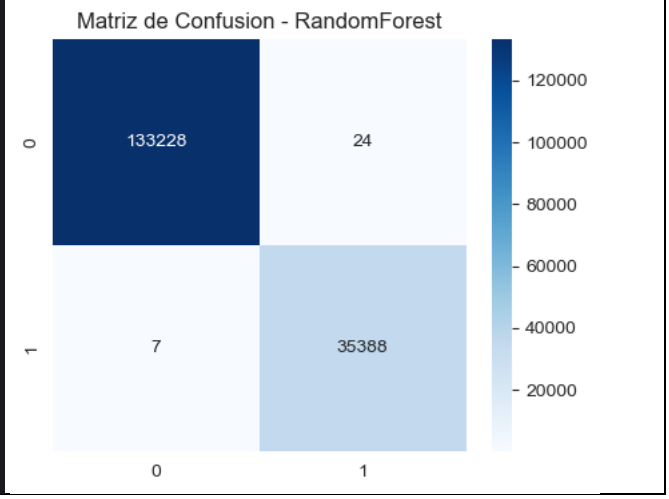
8 MODELO ML

El modelo de ML utilizado fue Random Forest, la versión de entrenamiento es la v3 ya que se realizaron ajustes y mejoras en el entrenamiento, así como en la limpieza de los datos de entrenamiento. Se configuró con el hiperparámetro n_estimators en un valor de 300. El modelo realiza una clasificación binaria como fraude/no fraude. Se implemento la validación StratifiedKFold de 5 particiones para el entrenamiento de esta versión.

8.1 MÉTRICAS DE DESEMPEÑO

Model	RandomForest
Accuracy	0.999816
Precision	0.999322
Recall	0.999802
F1	0.999562
ROC_AUC	0.999999
Specificity	0.99982
CV_ROC_AUC_MEAN	0.999999
CV_ROC_AUC_STD	0.0
threshold	0.37
intervalo_confianza	(0.999796, 1.000204)
margin_error	0.000204
standard_deviation	0.000164

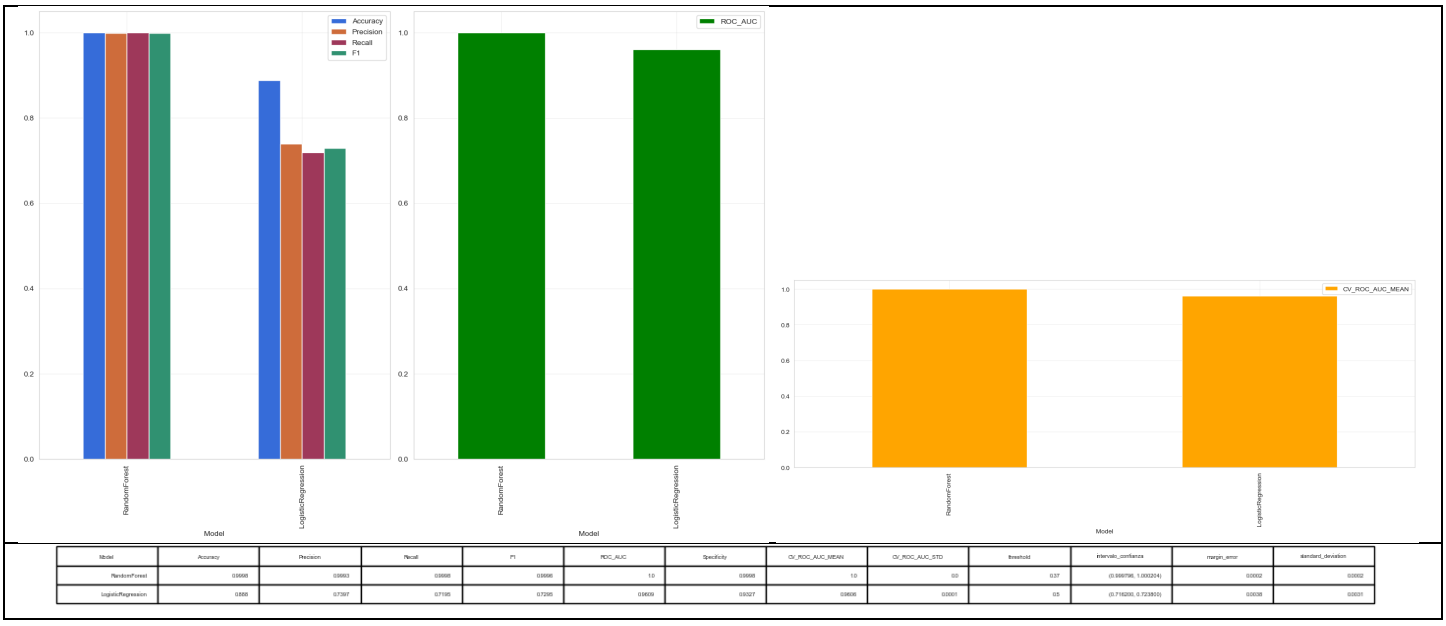
Name: 1, dtype: object

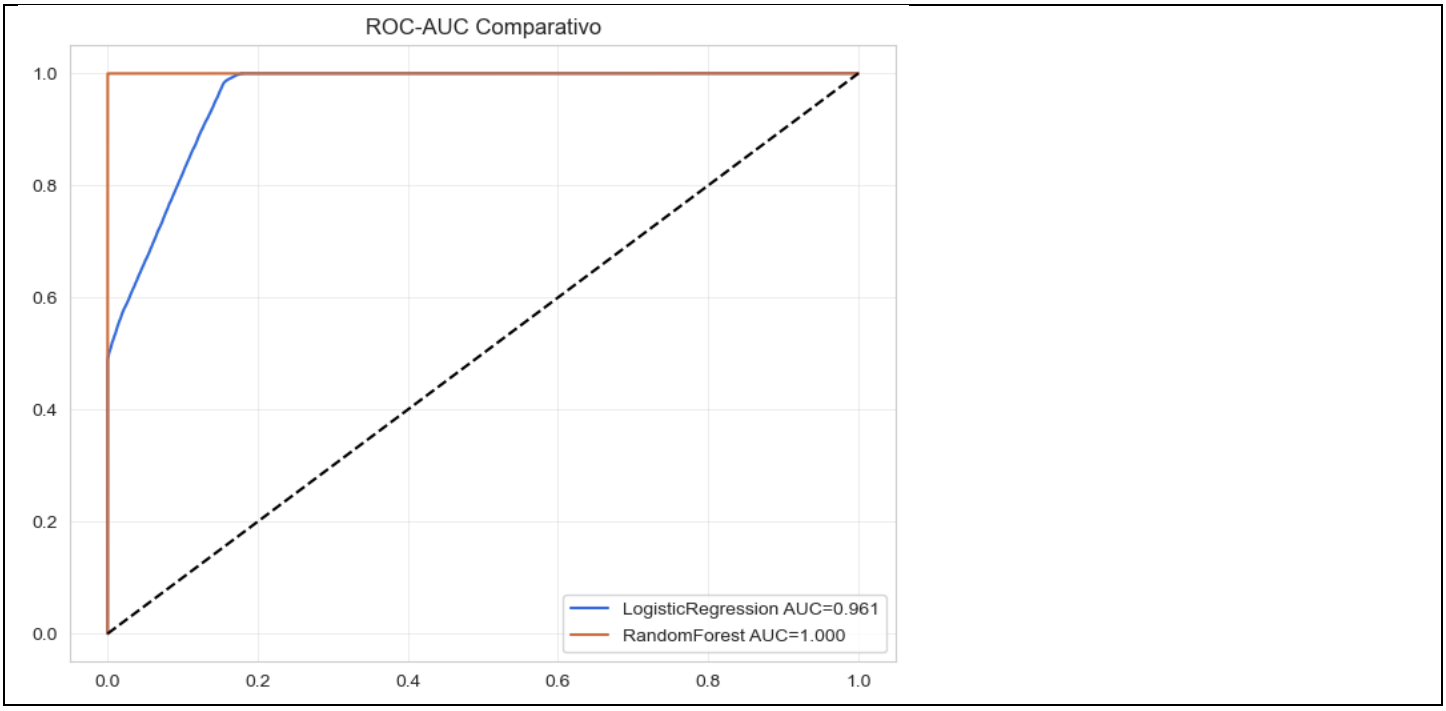


8.2 JUSTIFICACIÓN DEL UMBRAL

Se ejecutó búsqueda incremental de 0.01 optimizando simultáneamente F1 y Precisión. El valor seleccionado fue 0.37; ya que fue el que mejor desempeño obtuvo.

8.3 COMPARATIVA CON OTROS MODELOS.





9 PROCESO DE DESPLIEGUE

9.1 ESTRATEGIA DE DESPLIEGUE

Para el despliegue se usó una API con el Endpoint principal POST /predict; La respuesta JSON esperada: {"fraud": true}. Para este primer ejercicio se desplegó en un entorno local Dockerizado.

9.1.1 Tecnologías

Para un correcto funcionamiento del servicio se utilizaron las siguientes versiones de componentes y librerías: numpy 2.5.3, scikit-learn 1.9.1, fastapi 0.143.0, uvicorn 0.54.0, pandas 3.0.6.

9.1.2 Docker

El empaquetado de la solución se hizo mediante Dockerfile basado en python:3.13.16-slim.

- Construir la imagen de Docker: docker build -t <nombre> .
- Ejecutar el contenedor: docker run -p 8000:8000 <nombre>

9.1.3 Subir imagen a AWS

Para que AWS pueda ejecutar el contenedor, se debe subirlo a Amazon ECR (AWS, s.f.).

9.1.4 Desplegar en la Nube de AWS

Para desplegar en la nube se usará el servicio AWS App Runner (AWS, s.f.)

10 OPERACIÓN

Servicio continuo mediante Uvicorn. Revisión manual de errores HTTP.

10.1 MONITOREO

Logs generados por el servicio FastAPI/Uvicorn. No existe monitoreo automatizado durante el piloto.

10.2 Gobierno del Modelo

El área de Riesgos autoriza cambios de versión y actualizaciones.

10.3 Reentrenamiento

Se ejecuta nuevamente el pipeline completo cuando existan nuevos datos disponibles.

10.4 Seguridad

La solución utiliza HTTP y no implementa autenticación durante la fase piloto.

11 Riesgos

Dataset académico, ausencia de validación independiente, ausencia de calibración de probabilidades, falta de SHAP/LIME y potencial riesgo de data leakage u overfitting.

11.1 Limitaciones

Los resultados corresponden a un ejercicio académico y no deben considerarse evidencia suficiente para producción.

11.2 Recomendaciones

Implementar HTTPS, autenticación, monitoreo, trazabilidad, observabilidad y validación independiente.

11.3 Continuidad Operativa

Respaldar artefactos del modelo, código fuente y configuraciones del contenedor.

12 Conclusiones

El modelo cumple ampliamente los objetivos definidos para el piloto y demuestra viabilidad técnica.

13 Bibliografía

AWS. (s.f.). *Desplazamiento de una imagen a lo largo de su ciclo de vida en Amazon ECR* . Obtenido de Amazon ECR : https://docs.aws.amazon.com/es_es/AmazonECR/latest/userguide/getting-started-cli.html

AWS. (s.f.). *Getting started with App Runner* . Obtenido de AWS App Runner : https://docs.aws.amazon.com/apprunner/latest/dg/getting-started.html

ISO/IEC. (2022). *ISO/IEC 23053:2022*. Obtenido de ISO|: https://www.iso.org/standard/74438.html